Kernel Adventures: Part 2

29th Oct 2021 / Document No. D24.102.252

Prepared By: `clubby789`

Challenge Author(s): `clubby789`

Difficulty: Medium

Classification: Official

Synopsis

Description:

- Apparently Linux authentication is done in userspace? That doesn't sound safe, time to do it all in the kernel!

Objective:

- Analysing a new system call added to the kernel in order to abuse an integer overflow bug and gain a root shell.

Analysis

The patch adds a new system call to the kernel - `sys_magic`, with a syscall number of 449.

```
typedef enum {
    MAGIC_ADD = 0,
    MAGIC_EDIT = 1,
    MAGIC_DELETE = 2,
    MAGIC_SWITCH = 3
} MagicMode;
SYSCALL_DEFINE3(magic, MagicMode, mode, unsigned char __user*, username, unsigned
char __user*, password);
```

It also adds a spinlock to the kernel, enforcing exclusive access to the relevant data structures - it is locked when the syscall is entered, and unlocked when it returns.
As well as this, it adds a new `MagicUser` struct, and a global array of pointers to said users.

```
struct MagicUser {
    // The UID represented by this user
    kuid_t uid;
    // A pointer to a list of up to 64 pointers
    struct MagicUser** children;
    char username[64];
    char password[64];
};
```

The `static unsigned short nextId` is initialized to 0;

The syscall essentially implements a hierarchical authentication system. Upon the first call to sys_magic, the `root` user is initialized.
The general working of the system is explained below. NOTE: If any checks, allocations or accesses fail, the syscall returns a failure and undoes any changes. All string operations are properly bounded.

`MAGIC_ADD`

- The provided `username` and `password` are copied from userspace
- A check is performed to see if a user with the same name exists
- An empty slot is found within the `MAGIC_USERS` array
- The `MagicUser` tied to the current user is located
- An empty slot is located in the `children` array
- A new user is allocated, along with their own `children` array
- The user is given the ID `nextId`, and the provided username and password are copied into the buffers
- The pointer to the new user is placed in the current user's child array and the main array
- The user's ID is returned, and `nextId` is incremented

`MAGIC_EDIT`

- The current user is located in `magic_users`
- If the provided username matches the current user, then `child = currentUser`
- If not, each of the current user's children are checked - if any of their usernames match they are assigned to the `child` variable
- The provided password is copied into the user's password buffer

`MAGIC_SWITCH`

- The current user is located
- If the provided username is the same as the current user's, the syscall returns 0
- The provided username is searched for within the current user's children - if found, `child` is set to the found user
- If not found, the username is searched for within the global user list
- If the username is not found, the function fails
- If the user is the first in `magic_users` (i.e. root) it will fail

- If the password matches, `child` is set to the user (switching to a child of the current user doesn't require a password)
- A new credential struct is then allocated, using the UID from the `child`. Most of the code here is copied from `sys_setuid`
- The credential is committed to the current process

`MAGIC_DELETE`

- The current user is located
- The requested username is searched for in the current user's children
- If found, it will be deleted, and pointers to it in the child and global list removed
- This happens recursively, so all the user's children (and their children and so on) are properly deleted

Despite the length and apparent complexity of the code, there is a simple bug in the logic.
`nextId` is an unsigned short - every time a new user is created, it is incremented, with no range checks.
If `nextId` is `(1 << 16) - 1` (65535), then incrementing it will cause an integer overflow back to 0.
The correct behaviour here would either be to fail immediately, or to iterate the user list upon allocation to find an unused PID to use.
Using an `unsigned int` or `unsigned long` would also help to make this issue occur less often, as the range of UIDs would be orders of magnitude greater.

Exploitation

Exploitation of this bug is relatively simple. We simply need cause the `nextId` to increment until it reaches `USHRT_MAX`, then allocate another user and switch into them.
Their UID will be 0, and their credentials will be applied to the process, giving root privileges to the exploit.